

MPC INTERVIEW QUESTIONS & ANSWERS

Comprehensive Guide for Technical Interviews

TABLE OF CONTENTS

1. Fundamentals & Concepts
 2. Cost Function & Optimization
 3. Prediction & Horizon
 4. Constraints Handling
 5. Tuning & Parameters
 6. Comparison with Other Controllers
 7. Implementation & Practical
 8. Advanced Topics
 9. Real-World Applications
 10. Problem-Solving Questions
-

1 FUNDAMENTALS & CONCEPTS

Q1.1: What is Model Predictive Control (MPC)?

Answer: MPC is an advanced control strategy that uses a mathematical model to predict future system behavior and optimize control actions over a finite time horizon.

Key Points:

- Uses a **model** of the system dynamics
- **Predicts** future states over N steps (prediction horizon)
- **Optimizes** a cost function to find best control
- Applies **only the first control action** (receding horizon)
- **Repeats** the process at next time step

Mathematical Formulation:

At each time step k:

1. Measure current state: $x(k)$
2. Solve optimization problem:

minimize: $\sum_{i=0}^N (\|x(k+i) - x_{\text{target}}\|^2_Q + \|u(k+i)\|^2_R)$

subject to:

- $x(k+i+1) = f(x(k+i), u(k+i))$ [dynamics]
- $u_{\text{min}} \leq u(k+i) \leq u_{\text{max}}$ [control constraints]
- $x_{\text{min}} \leq x(k+i) \leq x_{\text{max}}$ [state constraints]

3. Apply $u^*(k)$ (first optimal control)
4. Repeat at $k+1$

Q1.2: What does "Receding Horizon" mean?

Answer: Receding horizon means MPC solves an optimization problem at each time step, but only implements the first control action from the optimal sequence, then re-solves at the next step with new measurements.

Analogy: Like driving a car:

- You look ahead (prediction horizon)
- Plan your steering (optimization)
- But you only turn the wheel NOW (first control action)
- Then you look ahead again from your new position (receding horizon)

Benefits:

- Handles disturbances naturally (feedback)
- Updates predictions with new information
- More robust than open-loop planning

Example:

Time $k=0$: Predict $k=0,1,2,3,4 \rightarrow$ Apply $u(0)$
Time $k=1$: Predict $k=1,2,3,4,5 \rightarrow$ Apply $u(1)$ ← horizon moved!
Time $k=2$: Predict $k=2,3,4,5,6 \rightarrow$ Apply $u(2)$

Q1.3: Why do we need a system model in MPC?

Answer: The model is used to **predict** how the system will evolve under different control actions. Without prediction, MPC cannot optimize future behavior.

Model Requirements:

1. **Accuracy:** Must capture important dynamics
2. **Computational efficiency:** Must evaluate quickly
3. **Differentiability:** Needed for gradient-based optimization

Types of Models:

- **Linear:** $x(k+1) = Ax(k) + Bu(k)$ - Fast, simple
- **Nonlinear:** $x(k+1) = f(x(k), u(k))$ - More accurate
- **Data-driven:** Learned from measurements (neural networks, etc.)

What if model is wrong?

- MPC is robust to small model errors (due to receding horizon)
 - Large errors → poor performance
 - Solution: Model adaptation, robust MPC, or learning-based MPC
-

2 COST FUNCTION & OPTIMIZATION

Q2.1: Explain the MPC cost function in detail.

Answer: The cost function defines what we want to minimize. It typically has three parts:

Standard Cost Function:

$$J = \sum_{k=0}^N (Q \cdot \|x(k) - x_{\text{ref}}\|^2 + R \cdot \|u(k)\|^2 + S \cdot \|\Delta u(k)\|^2)$$

Components:

1. **State Error Term:** $Q \cdot \|x(k) - x_{\text{ref}}\|^2$
 - Penalizes deviation from reference/target
 - Q is a positive-definite weighting matrix
 - Higher Q → track reference more aggressively
2. **Control Effort Term:** $R \cdot \|u(k)\|^2$

- Penalizes large control actions
- R is a positive-definite weighting matrix
- Higher $R \rightarrow$ smoother, less aggressive control

3. **Control Rate Term:** $S \cdot \|\Delta u(k)\|^2$ (optional)

- Penalizes rapid changes in control
- $\Delta u(k) = u(k) - u(k-1)$
- Helps avoid actuator wear

Example (Drone altitude):

```
python
cost = Q_z * (z - z_target)**2    # Want to reach target height
      + Q_v * v_z**2              # Want zero velocity at target
      + R * (F - F_hover)**2      # Minimize excess thrust
```

Q2.2: How do you choose Q and R weights?

Answer: This is a critical tuning question! There's no one-size-fits-all answer, but here are guidelines:

General Rules:

1. **Relative Magnitude:**

- Q/R ratio determines aggressiveness
- High $Q/R \rightarrow$ aggressive tracking, fast response
- Low $Q/R \rightarrow$ smooth control, slow response

2. **Starting Point:**

Start with: $Q = 100$, $R = 1$
Then tune based on performance

3. **Scaling:**

- Scale Q and R by state/control magnitudes
- If position is in meters, velocity in m/s, scale differently

$Q = \text{diag}([100, 10])$ # position vs velocity

Tuning Process:

Problem	Action
Too slow to reach target	$\uparrow Q$ or $\downarrow R$
Oscillations/overshoot	$\downarrow Q$ or $\uparrow R$
Too much control effort	$\uparrow R$
Not using full control authority	$\downarrow R$
Jerky motion	Add Δu term with high S

Trial Tuning:

1. Start: $Q=100$, $R=1$
2. If slow: $Q=200$, $R=1$
3. If oscillating: $Q=100$, $R=10$
4. Repeat until satisfied

LQR-based Method:

- Solve LQR for linear approximation
- Use resulting Q , R as starting point
- Fine-tune empirically

Q2.3: What optimization algorithm does MPC use?

Answer: Depends on the problem type and constraints:

For Linear MPC (quadratic cost, linear dynamics):

- **Quadratic Programming (QP)**
- Extremely fast and efficient
- Guaranteed global optimum
- Solvers: qpOASES, CVXGEN, OSQP

For Nonlinear MPC:

- **Sequential Quadratic Programming (SQP)**

- **Interior Point Methods**
- **Gradient Descent variants**
- Solvers: IPOPT, CasADi, ACADO

For Simple Systems (like our examples):

- **Grid Search** - Try many control values, pick best
- **Brute Force** - Exhaustive search
- Works well for 1-2 control dimensions

Real-time Considerations:

Computation time < Control period

Example:

Control period = 50ms

Optimization must finish in < 50ms

Q2.4: What is the difference between MPC cost function and reinforcement learning reward?

Answer:

MPC Cost Function:

- **Model-based:** Uses known dynamics
- **Lookahead:** Optimizes over finite horizon
- **Analytical:** Explicit mathematical form
- **Immediate:** Computed from model, not experience
- Example: $J = \sum (x - x_{\text{ref}})^2 + u^2$

RL Reward:

- **Model-free:** Learns from experience
- **Long-term:** Considers infinite horizon (with discount)
- **Learned:** Value function approximated
- **Cumulative:** Based on actual outcomes
- Example: $R = +10$ if goal, -1 for time, -100 if crash

Hybrid Approach (Model-Based RL):

- Use learned model in MPC framework
 - Combine advantages of both
-

3 PREDICTION & HORIZON

Q3.1: How do you choose the prediction horizon N?

Answer:

Key Factors:

1. System Dynamics Speed:

Fast system (motor): $N = 5-10$ steps
Slow system (temperature): $N = 20-50$ steps

2. Time Coverage:

Horizon should cover $1-3\times$ system settling time

If settling time = 2 seconds, $dt = 0.1s$
Then $N \geq 20$ steps

3. Computational Limit:

Longer $N \rightarrow$ More computation
Must solve in $<$ control period

4. Constraint Satisfaction:

N must be long enough to "see" upcoming constraints
Example: If obstacle is 1s away, need $N \times dt \geq 1s$

Practical Guidelines:

System Type	Typical N
Fast motor control	5-10
Robot navigation	10-20
Drone flight	15-30
Process control	20-100
Economic planning	50-200

Trade-off:

Short N: ✓ Fast computation ✗ Myopic behavior

Long N: ✓ Better foresight ✗ Slow computation

Q3.2: What happens if prediction horizon is too short?

Answer:

Problems:

1. Myopic Behavior:

- Can't "see" far enough ahead
- Makes locally optimal but globally poor decisions

Example:

Robot with N=2 steps (0.1s lookahead):

- Doesn't see wall 0.5s ahead
- Crashes even though it could have stopped

2. Can't Satisfy Future Constraints:

If speed limit zone is 1s away, but $N \times dt = 0.5s$

→ MPC doesn't know to start slowing down

3. Poor Setpoint Tracking:

System settling time = 3s, but $N \times dt = 1s$

- Controller can't plan proper trajectory
- Overshoot or slow convergence

4. Instability Risk:

For some systems, N too short → closed-loop unstable

Need $N \geq N_{\min}$ for stability guarantee

Real Example (Temperature Control):

System time constant: $\tau = 100s$

If $N = 5$, $dt = 1s$ → Only predicts 5s ahead

Problem: Can't anticipate heating needs for slow system

Solution: $N = 200$ (200s lookahead) for good performance

Q3.3: What is the difference between prediction horizon and control horizon?

Answer:

Prediction Horizon (N_p):

- How far ahead we **predict** states
- Always fixed

Control Horizon (N_c):

- How far ahead we **optimize** control
- Often $N_c < N_p$

Why use $N_c < N_p$?

1. Reduce Computation:

Instead of optimizing $u(0), u(1), \dots, u(N_p)$

Only optimize $u(0), u(1), \dots, u(N_c)$

Keep $u(N_c+1) = \dots = u(N_p) = u(N_c)$

2. Practical Benefit:

Prediction: $N_p = 20$ (look ahead 2s)

Control: $N_c = 5$ (only optimize first 0.5s)

Reduces optimization variables from 20 to 5

→ 4× faster computation!

Visualization:

Time: 0 1 2 3 4 5 6 7 8 9 10

↓ ↓ ↓ ↓ ↓

u: ? ? ? ? ? u5 u5 u5 u5 u5 ← $N_c = 5$

|-----|-----|

Optimize

Hold constant

|-----|

Predict all ($N_p = 10$)

4 CONSTRAINTS HANDLING

Q4.1: How does MPC handle constraints?

Answer:

Types of Constraints:

1. **Hard Constraints** (must be satisfied):

$u_{\min} \leq u(k) \leq u_{\max}$ [Control limits]

$x_{\min} \leq x(k) \leq x_{\max}$ [State limits]

$f(x(k), u(k)) \leq 0$ [General constraints]

2. **Soft Constraints** (prefer to satisfy):

Add slack variable ϵ :

$x(k) \leq x_{\max} + \epsilon$

Penalize: $J = \dots + \rho \cdot \epsilon^2$

Implementation Methods:

Method 1: Penalty Functions (Our approach)

python

```
def constraint_penalty(state, control):  
    penalty = 0  
  
    # Hard constraint: position limits  
    if state[0] < x_min:  
        penalty += 1000 * (x_min - state[0])**2  
    if state[0] > x_max:  
        penalty += 1000 * (state[0] - x_max)**2  
  
    # Hard constraint: control limits  
    if control[0] < u_min:  
        penalty += 1000 * (u_min - control[0])**2  
  
    return penalty  
  
cost = tracking_cost + control_cost + constraint_penalty
```

Method 2: Constrained Optimization

Use QP/NLP solver with explicit constraints:

```
minimize: J(x, u)  
subject to:  
-  $x_{\min} \leq x \leq x_{\max}$   
-  $u_{\min} \leq u \leq u_{\max}$   
-  $Ax + Bu \leq c$ 
```

Advantages of MPC for Constraints: ✓ Naturally handles constraints in optimization ✓ Can anticipate constraint violations ✓ Finds feasible solution if one exists ✓ Can prioritize constraints (soft vs hard)

Q4.2: What happens if constraints are infeasible?

Answer:

Infeasibility occurs when: No control sequence can satisfy all constraints over the horizon.

Example:

Current state: $x = 10\text{m}$, $v = 5\text{m/s}$ (moving fast)

Constraint: $x \leq 8\text{m}$ (wall at 8m)

Control limit: $|a| \leq 2 \text{ m/s}^2$

Physics: Need $a = -v^2/(2 \cdot \Delta x) = -25/4 = -6.25 \text{ m/s}^2$

But constraint: $|a| \leq 2 \text{ m/s}^2$

→ INFEASIBLE! Can't avoid wall with available control.

Solutions:

1. Soft Constraints:

python

Allow constraint violation with penalty

`cost += M * max(0, x - x_max)**2`

where M is very large but *not* ∞

2. Constraint Relaxation:

Instead of: $x \leq 8$

Use: $x \leq 8 + \epsilon$, penalize ϵ

3. Emergency Behavior:

python

if optimization_infeasible:

Fall back to safe action

`u = emergency_stop()`

4. Tighten Constraints Proactively:

Use $x \leq 7.5\text{m}$ instead of $x \leq 8\text{m}$

→ More conservative, avoids infeasibility

5. Reduce Horizon:

If N=10 infeasible, try N=5

Shorter horizon may have feasible solution

Q4.3: Give an example of state constraints vs control constraints.

Answer:

Control Constraints (what actuator can do):

Example 1: Motor Voltage

$$-12V \leq V_{\text{motor}} \leq 12V$$

Reason: Hardware limit - motor driver is 12V

Example 2: Thrust

$$0N \leq F_{\text{thrust}} \leq 25N$$

Reason: Physical limit - motor can't produce more

Example 3: Steering Angle

$$-30^\circ \leq \delta \leq 30^\circ$$

Reason: Mechanical limit - steering mechanism

State Constraints (where system can be):

Example 1: Altitude

$$0m \leq z \leq 100m$$

Reason:

- Can't go below ground ($z \geq 0$)
- Regulatory limit ($z \leq 100$)

Example 2: Velocity

$$-5 \text{ m/s} \leq v \leq 5 \text{ m/s}$$

Reason:

- Safety limit
- Model validity range

Example 3: Temperature

$$60^\circ\text{C} \leq T \leq 80^\circ\text{C}$$

Reason:

- Too cold: product quality suffers ($T \geq 60$)
- Too hot: equipment damage ($T \leq 80$)

Key Difference:

- **Control constraints:** Direct limits on what we command
 - **State constraints:** Limits on where system can go (must predict to enforce!)
-

5 TUNING & PARAMETERS

Q5.1: Walk me through tuning an MPC controller step-by-step.

Answer:

Step-by-Step Tuning Process:

Step 1: Set Physical Parameters

```
python

# Measure/identify these first
m = 1.0 kg      # mass (weigh it!)
g = 9.81 m/s²   # gravity (known)
dt = 0.05 s     # control period (your choice)
```

Step 2: Choose Initial Horizon

```
python
```

```
N = 10 # Start with 10 steps
```

```
# Rule:  $N * dt$  should cover 1-3× settling time
```

```
# If system settles in 2s,  $N*dt \approx 1-2s$ 
```

Step 3: Set Initial Weights (Conservative)

```
python
```

```
Q = 1.0 # Low tracking weight (gentle)
```

```
R = 1.0 # Medium control weight
```

Step 4: Test Baseline

```
Run simulation, observe:
```

- Does it reach target? (if not → increase Q)
- How fast? (too slow → increase Q, too fast → decrease Q)
- Smooth control? (jerky → increase R)

Step 5: Increase Tracking Performance

```
python
```

```
Q = 10.0 # Increase by 10×
```

```
# Test again
```

```
# Still not aggressive enough?
```

```
Q = 100.0 # Increase more
```

Step 6: Adjust Smoothness

```
python
```

```
# If control is too aggressive/jerky
```

```
R = 10.0 # Increase R
```

```
# If control is too timid
```

```
R = 0.1 # Decrease R
```

Step 7: Fine-tune Horizon

```
python
```

If missing constraints or poor long-term planning

N = 20 # Increase horizon

If computation too slow

N = 5 # Decrease horizon

Step 8: Add Constraint Penalties (if needed)

python

If violating constraints

penalty_weight = 1000.0 # Large penalty

Example Tuning Session:

Iteration 1: Q=1, R=1, N=10

→ Too slow (takes 10s to reach target)

Iteration 2: Q=100, R=1, N=10

→ Fast but oscillates

Iteration 3: Q=100, R=10, N=10

→ Good speed, but still small oscillations

Iteration 4: Q=100, R=20, N=10

→ Perfect! ✓

Q5.2: How do you know if your MPC is well-tuned?

Answer:

Performance Metrics:

1. Settling Time:

Time to reach and stay within $\pm 5\%$ of target

Target: $< 5 \times$ system time constant

2. Overshoot:

Maximum deviation beyond target

Target: < 10% for position control

3. Steady-State Error:

Final error after settling

Target: < 1% of reference

4. Control Effort:

$\sum u(k)^2$

Should not saturate (hitting limits constantly)

Should not be tiny (not using available authority)

5. Tracking Error:

$\text{RMS error} = \sqrt{(\sum (x - x_{\text{ref}})^2 / N)}$

Lower is better

Visual Indicators:

✓ Well-tuned:

Position:  (smooth approach to target)

Control:  (ramps up then down)

✗ Poorly tuned (Q too high):

Position:  (oscillations)

Control:  (chattering)

✗ Poorly tuned (Q too low):

Position:  (too slow)

Control:  (gentle, takes forever)

Quantitative Test:

python

```

def evaluate_tuning(time_data, position_data, control_data, target):
    # 1. Settling time
    settled = np.abs(position_data - target) < 0.05 * target
    settling_idx = np.argmax(settled)
    settling_time = time_data[settling_idx]

    # 2. Overshoot
    overshoot = (np.max(position_data) - target) / target * 100

    # 3. RMS error
    rms_error = np.sqrt(np.mean((position_data - target)**2))

    # 4. Control effort
    control_effort = np.sum(control_data**2)

    print(f"Settling time: {settling_time:.2f}s")
    print(f"Overshoot: {overshoot:.1f}%")
    print(f"RMS error: {rms_error:.3f}")
    print(f"Control effort: {control_effort:.1f}")

    # Good tuning criteria
    good = (settling_time < 5.0 and
            overshoot < 10 and
            rms_error < 0.1)

    return good

```

Q5.3: What's the difference between tuning MPC vs PID?

Answer:

PID Tuning:

Parameters: Kp, Ki, Kd (3 parameters)
 Focus: Response characteristics (rise time, overshoot, settling)
 Methods: Ziegler-Nichols, Cohen-Coon, trial & error

MPC Tuning:

Parameters: Q, R, N (and more) (3+ parameters)
Focus: Optimization objective (what to minimize)
Methods: LQR theory, trial & error, systematic testing

Key Differences:

Aspect	PID	MPC
Intuition	K_p = proportional gain	Q = how much you care about error
	K_d = derivative gain	R = how much you care about control
Constraints	Can't handle directly	Natural incorporation
Multivariable	SISO (needs separate PIDs)	MIMO (handles all together)
Prediction	No lookahead	Explicit lookahead (N)
Complexity	Simple, fast	Complex, slower

Tuning Analogy:

PID: Like adjusting your driving reflexes

- K_p : How hard you react to being off course
- K_i : How much you remember past errors
- K_d : How much you anticipate future errors

MPC: Like planning your route

- Q: How important is it to stay on route?
- R: How important is it to drive smoothly?
- N: How far ahead do you plan?

6 COMPARISON WITH OTHER CONTROLLERS

Q6.1: When should you use MPC instead of PID?

Answer:

Use PID when:

- ✓ Simple SISO system (one input, one output)
- ✓ Fast dynamics (millisecond response needed)
- ✓ Linear system near operating point
- ✓ No hard constraints
- ✓ Low computational resources
- ✓ Easy to tune is critical

Examples: Motor speed control, temperature regulation, cruise control

Use MPC when:

- ✓ MIMO system (multiple inputs/outputs)
- ✓ Hard constraints must be satisfied
- ✓ Predictable disturbances
- ✓ Long time constants (slower dynamics)
- ✓ Optimal performance critical
- ✓ Computational resources available

Examples: Drone flight, robot path following, chemical processes, HVAC

Concrete Comparison:

Problem: Altitude Control

PID Approach:

```
python

error = z_target - z_current
control = Kp*error + Ki*ferror + Kd*d(error)/dt
```

Issues:

- Might violate thrust limits
- No prediction of future position
- Can't handle velocity constraints

MPC Approach:

```
python
```

control = optimize(predict N steps ahead)

subject to: $0 \leq F \leq F_{\max}$

$-v_{\max} \leq v \leq v_{\max}$

Benefits:

- Respects all limits
- Predicts trajectory
- Optimizes path to target

Q6.2: What are the advantages and disadvantages of MPC?

Answer:

Advantages:

1. Handles Constraints Naturally

Can explicitly include:

- Actuator limits
- Safety bounds
- Regulatory limits

2. Multivariable

Controls multiple inputs to achieve multiple outputs

Example: Drone uses 4 motors to control x, y, z, yaw

3. Optimal

Minimizes cost function

Finds best trajectory, not just any trajectory

4. Predictive

Anticipates future events

Can handle known disturbances

5. Flexible

Easy to change objectives (just modify cost function)
Can handle time-varying references

Disadvantages:

1. Computational Cost

Optimization at every time step
May need powerful computer for fast systems

2. Requires Model

Performance depends on model accuracy
Bad model → bad control

3. Tuning Complexity

More parameters than PID
Less intuitive for non-experts

4. No Stability Guarantee (for nonlinear)

Linear MPC: Can prove stability
Nonlinear MPC: Harder to guarantee

5. Implementation Complexity

Harder to implement than PID
Requires optimization library

When MPC is Worth It:

Benefits > Costs when:

- Constraints are critical
- System is expensive (optimal operation valuable)
- Multiple objectives must be balanced
- Computation time is acceptable

Q6.3: Can you combine MPC with other controllers?

Answer:

Yes! **Hybrid architectures** are common:

Architecture 1: MPC + Low-Level PID

High Level (slow): MPC → reference trajectories

Low Level (fast): PID → track references

Example: Quadcopter

- MPC (10 Hz): Computes desired position/velocity
- PID (100 Hz): Controls motor speeds to achieve it

Architecture 2: MPC + Feedforward

MPC → optimal control

Feedforward → compensate known disturbances

Example: Robot arm

- Feedforward: Gravity compensation
- MPC: Position tracking

Total control = $u_{\text{mpc}} + u_{\text{ff}}$

Architecture 3: MPC with RL

RL: Learn system model

MPC: Use learned model for control

Example: Complex robot

- RL learns dynamics from data
- MPC uses learned model to plan

Architecture 4: Switching Controllers

```
if (near_target):  
    use PID # Fast, simple  
else:  
    use MPC # Optimal approach
```

Example: Drone landing

- MPC for approach phase
- PID for final hovering

Code Example:

```
python  
  
class HybridController:  
    def __init__(self):  
        self.mpc = MPC_Controller(...)  
        self.pid = PID_Controller(Kp=1, Ki=0.1, Kd=0.05)  
  
    def compute_control(self, state, target):  
        # MPC for trajectory planning  
        reference = self.mpc.compute_setpoint(state, target)  
  
        # PID for tracking  
        control = self.pid.update(state, reference)  
  
    return control
```

7 IMPLEMENTATION & PRACTICAL

Q7.1: How do you implement MPC in real-time?

Answer:

Key Challenges:

1. Optimization must complete before next control cycle
2. Must handle real sensor noise
3. Must be robust to model errors

Real-Time Implementation Strategy:

Step 1: Choose Appropriate dt

```
python
```

```
# Control period must be achievable  
dt = 0.05 # 50ms = 20 Hz control rate  
  
# Rule: optimization_time < 0.8 × dt  
# Leave margin for other tasks
```

Step 2: Limit Optimization Complexity

```
python
```

```
# Use shorter horizon for faster computation  
N = 10 # Not 100!  
  
# Reduce control sampling  
n_samples = 15 # Not 1000!  
  
# Use warm starting (initialize with previous solution)  
u_init = u_previous
```

Step 3: Implement Timing Protection

```
python
```

```
import time

def control_loop():
    while True:
        start_time = time.time()

        # Read sensors
        state = read_sensors()

        # Compute MPC with timeout
        try:
            control = mpc.compute_control(state, target, timeout=0.04)
        except TimeoutError:
            # Fall back to previous control if too slow
            control = previous_control

        # Apply control
        send_to_actuators(control)

        # Timing check
        elapsed = time.time() - start_time
        if elapsed > dt:
            print(f"WARNING: Control loop slow ({elapsed:.3f}s > {dt}s)")

        # Wait for next cycle
        time.sleep(max(0, dt - elapsed))
```

Step 4: Handle Sensor Noise

```
python
```

```

# Use filtering
from scipy.signal import butter, filtfilt

class MPCWithFiltering:
    def __init__(self):
        self.mpc = MPC_Controller(...)
        # Low-pass filter for noisy measurements
        self.b, self.a = butter(2, 0.1) # 2nd order, cutoff=0.1
        self.state_history = []

    def filter_state(self, raw_state):
        self.state_history.append(raw_state)
        if len(self.state_history) > 10:
            self.state_history.pop(0)

        # Filter if enough history
        if len(self.state_history) >= 5:
            filtered = filtfilt(self.b, self.a,
                               np.array(self.state_history), axis=0)
            return filtered[-1]
        return raw_state

```

Step 5: Add Safety Checks

```

python

def safe_control_loop():
    while True:
        state = read_sensors()

        # Safety check
        if is_unsafe(state):
            control = emergency_stop()
        else:
            control = mpc.compute_control(state, target)
            control = saturate(control, min_u, max_u) # Clip limits

        send_to_actuators(control)
        sleep(dt)

def is_unsafe(state):
    return (state[0] < safety_bound_min or
            state[0] > safety_bound_max)

```

Q7.2: How do you validate an MPC controller before deploying?

Answer:

Validation Process:

Level 1: Simulation Testing

```
python

# Test in ideal simulation
results = simulate_mpc(initial_state, target, duration=10.0)

# Check performance
assert results['settling_time'] < 5.0
assert results['overshoot'] < 0.1
assert results['final_error'] < 0.01
```

Level 2: Robustness Testing

```
python

# Test with model uncertainty
for model_error in [0.9, 1.0, 1.1]: # ±10% mass error
    model_perturbed = modify_model(nominal_model, model_error)
    results = simulate_mpc(model_perturbed, ...)
    assert results['stable']

# Test with disturbances
for disturbance_level in [0.1, 0.5, 1.0]:
    results = simulate_with_disturbance(disturbance_level)
    assert results['recovers']
```

Level 3: Edge Case Testing

```
python
```

```
test_cases = [  
    ('Zero initial velocity', [0, 0, 0], [10, 0, 0]),  
    ('High initial velocity', [0, 5, 0], [10, 0, 0]),  
    ('Negative target', [10, 0, 0], [-5, 0, 0]),  
    ('Constraint violation', [15, 0, 0], [20, 0, 0]), # Beyond max  
]  
  
for name, init, target in test_cases:  
    results = simulate_mpc(init, target)  
    assert results['safe'], f"Failed: {name}"
```

Level 4: Hardware-in-Loop (HIL)

```
python  
  
# Run MPC on real-time simulator with actual hardware  
hil_system = HardwareInLoop(actual_sensors, actual_actuators)  
  
for scenario in scenarios:  
    response = hil_system.run(mpc, scenario)  
    validate(response)
```

Level 5: Incremental Deployment

1. Test stationary (motors off, just compute control)
2. Test with constraints (limited speed/range)
3. Test with safety envelope (restricted zone)
4. Test full operation (gradual expansion)

Checklist:

- ☐ Simulation: All scenarios pass
- ☐ Robustness: Works with $\pm 20\%$ parameter error
- ☐ Timing: Computation < 80% of control period
- ☐ Safety: Emergency stop works
- ☐ Edge cases: Handles constraint violations gracefully
- ☐ HIL: Real hardware responds correctly
- ☐ Documentation: All parameters recorded

Q7.3: What are common implementation bugs in MPC?

Answer:

Bug 1: Wrong Sign in Dynamics

```
python

# WRONG
az = F/m + g # Gravity adds to thrust?!

# CORRECT
az = F/m - g # Thrust opposes gravity
```

Bug 2: Not Checking Constraint Violations

```
python

# WRONG: Just optimize, ignore if infeasible
control = optimize(...)

# CORRECT: Check and handle
try:
    control = optimize(...)
except InfeasibleError:
    control = fallback_control()
```

Bug 3: Forgetting to Update State

```
python

# WRONG: State never changes!
while True:
    control = mpc.compute(state, target) # Same state every time!
    apply(control)

# CORRECT: Update state
while True:
    control = mpc.compute(state, target)
    apply(control)
    state = measure_new_state() # Update!
```

Bug 4: Mismatched Units

```
python
```

```
# WRONG
dt = 50 # Milliseconds? Seconds?
N = 10
horizon_time = N * dt # 500 what?!

# CORRECT
dt = 0.050 # Seconds (explicit)
N = 10 # Steps
horizon_time = N * dt # 0.5 seconds (clear)
```

Bug 5: Not Saturating Control

```
python

# WRONG: Might command impossible values
control = mpc.compute(...)
apply(control) # What if control = 1000V on 12V motor?

# CORRECT: Saturate to limits
control = mpc.compute(...)
control = np.clip(control, u_min, u_max)
apply(control)
```

Bug 6: Division by Zero

```
python

# WRONG: What if self.R = 0?
omega = np.sqrt(thrust / self.k_f) # sqrt(F/0) = ∞

# CORRECT: Check for zero
if self.k_f > 0:
    omega = np.sqrt(max(0, thrust) / self.k_f)
else:
    omega = 0
```

Bug 7: Infinite Loop in Optimization

```
python
```

```
# WRONG: No iteration limit
while not converged:
    # optimize...
    # What if never converges?

# CORRECT: Max iterations
for iter in range(max_iterations):
    if converged:
        break
    # optimize...
if not converged:
    print("Warning: Did not converge")
```

8 ADVANCED TOPICS

Q8.1: What is the difference between linear and nonlinear MPC?

Answer:

Linear MPC:

Dynamics: $x(k+1) = Ax(k) + Bu(k)$
Cost: Quadratic ($x'Qx + u'Ru$)
Optimization: Quadratic Programming (QP)

Properties:

- ✓ Globally optimal solution
- ✓ Fast computation (milliseconds)
- ✓ Stability guarantees available
- ✓ Well-established theory
- ✗ Only valid near operating point
- ✗ Can't capture nonlinear effects

Example (Drone near hover):

python

```
# Linear approximation around hover
A = [[1, dt],
      [0, 1]] # Position-velocity model
B = [[0],
      [dt/m]] # Control affects acceleration

# Valid only for small deviations from hover!
```

Nonlinear MPC:

```
Dynamics:  $x(k+1) = f(x(k), u(k))$  [arbitrary function]
Cost: General  $J(x, u)$ 
Optimization: Nonlinear Programming (NLP)
```

Properties:

- ✓ Accurate over large operating range
- ✓ Captures true system behavior
- ✓ Better performance
- ✗ Slower computation (10-100× slower)
- ✗ Local optima possible
- ✗ Harder stability analysis

Example (Drone full dynamics):

```
python

def dynamics(state, control):
    x, y, z, vx, vy, vz, roll, pitch, yaw = state
    F, tau_roll, tau_pitch, tau_yaw = control

    # Full nonlinear kinematics
    ax = (sin(pitch)*cos(roll)*cos(yaw) + ...) * F/m
    # ... complex trigonometry

    return next_state # No simple A, B matrices!
```

When to use which:

Condition	Choice
Small deviations from setpoint	Linear MPC
Large maneuvers	Nonlinear MPC
Real-time critical ($< 1\text{ms}$)	Linear MPC
Accuracy critical	Nonlinear MPC
System naturally linear	Linear MPC
Strong nonlinearities	Nonlinear MPC

Q8.2: What is offset-free tracking in MPC?

Answer:

Problem: Steady-State Offset

Even if MPC converges, there might be a constant error:

Target: 10m
Actual: 9.8m (close but not exact)

Causes:

1. Model mismatch (e.g., actual mass $\neq$ model mass)
2. Unmodeled disturbances (e.g., wind, friction)
3. Actuator bias

Solution 1: Add Integral Action

```
python
# Like PID's integral term
error_integral += (target - state) * dt

cost = Q*(state - target)2 + QI*error_integral2 + R*control2
```

Solution 2: Disturbance Observer

```
python
```

```
# Estimate constant disturbance
```

```
d_hat = (state_measured - state_predicted) / dt
```

```
# Include in prediction
```

```
state_next = f(state, control) + d_hat
```

Solution 3: Target Calculator

```
python
```

```
# Find steady-state control that achieves target exactly
```

```
# Solve:  $x_{ss} = f(x_{ss}, u_{ss})$  with  $x_{ss} = target$ 
```

```
u_steady_state = solve_for_steady_state(target)
```

```
# Use as reference in cost
```

```
cost = Q*(state - target)2 + R*(control - u_steady_state)2
```

Implementation Example:

```
python
```

```

class MPCWithIntegral:
    def __init__(self):
        self.error_integral = 0
        self.Q_i = 10.0

    def compute_control(self, state, target):
        # Update integral
        error = target - state
        self.error_integral += error * self.dt

        # Optimize with integral term
        best_cost = float('inf')
        for control in candidates:
            predicted = self.predict(state, control, self.N)
            cost = sum(Q*(p - target)**2 for p in predicted)
            cost += self.Q_i * self.error_integral**2 # Integral term
            cost += R * control**2

            if cost < best_cost:
                best_control = control
                best_cost = cost

        return best_control

```

Q8.3: What is economic MPC?

Answer:

Traditional MPC: Minimize tracking error:

$$J = \sum ||x(k) - x_{ref}||^2$$

Goal: Follow a reference trajectory

Economic MPC: Minimize economic cost:

$$J = \sum \text{economic_cost}(x(k), u(k))$$

Goal: Optimize profit, energy, etc.

Example 1: Power Plant

```
python
```

```
# Traditional MPC: Track power setpoint
```

```
cost = Q * (power - power_ref)**2
```

```
# Economic MPC: Maximize profit
```

```
cost = -electricity_price * power + fuel_cost(fuel_rate) + wear_cost(control2)
```

Example 2: Building HVAC

```
python
```

```
# Traditional MPC: Track temperature setpoint
```

```
cost = Q * (T - T_ref)**2 + R * power**2
```

```
# Economic MPC: Minimize energy bill
```

```
cost = electricity_price(time) * power  
      + comfort_penalty(T - T_desired)  
      + demand_charge(peak_power)
```

Example 3: Warehouse Robot

```
python
```

```
# Traditional MPC: Follow path
```

```
cost = Q * (pos - path)**2
```

```
# Economic MPC: Minimize total operation cost
```

```
cost = time_to_delivery * delivery_cost_per_second  
      + energy_use * electricity_cost  
      + motor_wear(control2)
```

Key Difference:

- Traditional: Reference → Control
- Economic: Economics → Control (no reference needed!)

When to use:

- Process industries (chemical, power)
- Building energy management
- Supply chain optimization
- Any system where cost matters more than tracking

9 REAL-WORLD APPLICATIONS

Q9.1: Describe MPC application in autonomous vehicles.

Answer:

Autonomous Vehicle MPC Architecture:

Level 1: Path Planning (Global)

```
Start → A* / RRT → Waypoints → Feed to MPC
```

Level 2: MPC Control (Local)

State:

```
python
x = [x, y, θ, v] # Position, heading, velocity
```

Control:

```
python
u = [δ, a] # Steering angle, acceleration
```

Dynamics (Bicycle Model):

```
python
dx/dt = v * cos(θ)
dy/dt = v * sin(θ)
dθ/dt = (v/L) * tan(δ)
dv/dt = a
```

Cost Function:

```
python
```

```

J = Σ[k=0 to N] (
    Q_path * distance_to_path²      # Follow planned path
+ Q_obstacle * obstacle_penalty    # Avoid obstacles
+ Q_speed * (v - v_desired)²      # Maintain speed
+ R_steer * δ²                     # Minimize steering
+ R_accel * a²                     # Smooth acceleration
+ S * (δ(k) - δ(k-1))²           # Smooth steering changes
)

```

Constraints:

```

python

# Steering limits
-30° ≤ δ ≤ 30°

# Acceleration limits
-8 m/s² ≤ a ≤ 3 m/s² # Brake vs accelerate

# Speed limits
0 ≤ v ≤ speed_limit(location)

# Safety: Collision avoidance
distance_to_obstacles ≥ safety_margin

```

Real-World Challenges:

1. Computation Time:

Need solution in < 100ms for 10 Hz control
 Solution: Use linear MPC or code generation

2. Sensor Uncertainty:

Obstacle positions are noisy
 Solution: Robust MPC or chance-constrained MPC

3. Unknown Disturbances:

Wind, road slope unknown

Solution: Adaptive MPC, disturbance observer

Example Companies Using MPC:

- Tesla (Autopilot path following)
- Waymo (Motion planning)
- Cruise (Trajectory optimization)

Q9.2: How is MPC used in drones?

Answer:

Drone MPC Hierarchy:

Outer Loop (Position MPC):

Control frequency: 10-20 Hz

State: $[x, y, z, v_x, v_y, v_z]$

Control: $[F_{\text{total}}, \phi_{\text{desired}}, \theta_{\text{desired}}, \psi_{\text{rate}}]$
(thrust, roll, pitch, yaw rate)

Inner Loop (Attitude PID/MPC):

Control frequency: 100-500 Hz

State: $[\phi, \theta, \psi, p, q, r]$

(roll, pitch, yaw, rates)

Control: [Motor 1, Motor 2, Motor 3, Motor 4]

Position MPC Cost Function:

python

```
J = Σ (  
    Q_pos * ||pos - pos_target||2    # Position tracking  
    + Q_vel * ||vel||2                # Minimize velocity at target  
    + R_thrust * (F - F_hover)2        # Minimize excess thrust  
    + R_tilt * (φ2 + θ2)              # Minimize tilt  
)
```

Constraints:

python

Tilt limits (can't tilt more than 45°)

$-45^\circ \leq \phi, \theta \leq 45^\circ$

Thrust limits

$0 \leq F \leq F_{\max}$

Speed limits

$\|v\| \leq v_{\max}$

Altitude limits

$0.5\text{m} \leq z \leq 120\text{m}$ *# FAA regulations*

Advanced: Trajectory Optimization

python

Not just reach target, but follow a trajectory

for k in range(N):

pos_ref = trajectory(t + k*dt)

cost += Q * ||pos(k) - pos_ref||²

Handling Wind:

python

Estimate wind disturbance

wind_est = (v_measured - v_predicted)

Include in prediction

v_next = v + a*dt + wind_est*dt

Collision Avoidance:

python

Add constraint for each obstacle i

for obstacle in obstacles:

distance = ||pos - obstacle.pos||

if distance < safe_distance:

penalty += 1000 * (safe_distance - distance)²

Practical Implementation (ROS):

python

```
class DroneMPC:
    def __init__(self):
        self.sub = rospy.Subscriber('/mavros/local_position/pose', ...)
        self.pub = rospy.Publisher('/mavros/setpoint_raw/attitude', ...)

    def position_callback(self, msg):
        pos = [msg.pose.position.x, y, z]
        vel = estimate_velocity(pos)

        # MPC computes desired thrust and attitude
        F, roll, pitch, yaw_rate = self.mpc.compute_control(pos, vel, target)

        # Publish to inner loop controller
        self.publish_setpoint(F, roll, pitch, yaw_rate)
```

Q9.3: What industries use MPC most?

Answer:

Industry Rankings by MPC Adoption:

1. Chemical & Petrochemical (★★★★★)

Applications:

- Distillation columns
- Reactors
- Refineries

Why MPC:

- Complex MIMO systems
- Tight constraints (safety, quality)
- Economic optimization critical
- Slow dynamics (minutes to hours)

Market: Largest MPC deployment worldwide

Companies: Honeywell, Aspen Tech, Yokogawa

2. Oil & Gas (★★★★★)

Applications:

- Pipeline networks
- Gas processing
- Offshore platforms

Why MPC:

- Regulatory constraints
- Environmental limits
- Production optimization
- Handling disturbances

Example: North Sea platforms save millions with MPC

3. Power Generation (★★★★☆)

Applications:

- Gas turbines
- Steam cycles
- Combined cycle plants

Why MPC:

- Efficiency optimization
- Emission limits
- Load following
- Economic dispatch

Benefit: 1-2% efficiency = millions in fuel savings

4. Automotive (★★★★☆)

Applications:

- Adaptive cruise control
- Lane keeping
- Autonomous driving
- Engine control

Why MPC:

- Predictive planning
- Constraint handling
- Optimal fuel economy

Companies: Tesla, BMW, Ford, Toyota

5. Aerospace (★★★★☆)

Applications:

- Aircraft flight control
- Spacecraft attitude control
- Drone navigation
- Missile guidance

Why MPC:

- Complex dynamics
- Strict safety constraints
- Optimal fuel usage

Example: SpaceX uses MPC for rocket landing

6. Robotics (★★★★☆)

Applications:

- Manipulator control
- Mobile robots
- Warehouse automation
- Surgical robots

Why MPC:

- Collision avoidance
- Multi-DOF coordination
- Optimal trajectories

Growing field with increasing adoption

7. Building HVAC (★★★★☆)

Applications:

- Heating/cooling optimization
- Demand response
- Energy storage
- Comfort control

Why MPC:

- Energy cost minimization
- Thermal comfort
- Peak shaving
- Renewable integration

Example: Google data centers use MPC (30% cooling savings)

8. Renewable Energy (★★★★☆☆)

Applications:

- Wind farm control
- Solar tracking
- Battery management
- Grid integration

Why MPC:

- Weather prediction
- Economic optimization
- Grid constraints

Fast-growing application area

Revenue by Industry (Approximate):

Chemical/Oil & Gas: 60%

Power & Energy: 15%

Manufacturing: 10%

Automotive: 8%

Aerospace: 4%

Others: 3%

10 PROBLEM-SOLVING QUESTIONS

Q10.1: Your MPC is oscillating. Debug it step-by-step.

Answer:

Step 1: Identify Type of Oscillation

Observation 1: High-frequency oscillation (chattering)

Time scale: Every control step

Cause: Likely Q too high or R too low

Observation 2: Low-frequency oscillation (hunting)

Time scale: Multiple seconds

Cause: Likely wrong model parameters or N too short

Step 2: Check Parameters

```
python

# Print current parameters
print(f"Q = {Q}, R = {R}, N = {N}, dt = {dt}")

# Check ratios
print(f"Q/R ratio = {Q/R}") # If > 1000, too aggressive

# Check horizon time
print(f"Horizon = {N*dt}s") # Should be 1-3× settling time
```

Step 3: Reduce Aggressiveness

```
python

# Try cutting Q in half
Q_new = Q / 2

# Or increasing R by 10×
R_new = R * 10

# Test with new parameters
results = test_mpc(Q_new, R_new)
```

Step 4: Check Model Accuracy

```
python

# Compare predicted vs actual
for k in range(100):
    predicted_state = model.predict(state, control, dt)
    apply_control(control)
    time.sleep(dt)
    actual_state = measure_state()

    error = np.linalg.norm(predicted_state - actual_state)
    print(f"Model error: {error}")

    if error > 0.5: # Large error
        print("Model inaccurate! Check parameters:")
        print("- Mass correct?")
        print("- Time constant correct?")
        print("- Friction modeled?")
```

Step 5: Add Damping

```
python

# Option 1: Increase velocity penalty
Q_vel = Q_vel * 5 # Penalize velocity more

# Option 2: Add control rate penalty
S = 10.0 # Penalize Δu
cost += S * (u[k] - u[k-1])**2

# Option 3: Low-pass filter control
u_filtered = 0.7 * u_previous + 0.3 * u_new
```

Step 6: Check Constraints

```
python

# Are we hitting constraints every step?
if control == control_max or control == control_min:
    print("Saturating! This can cause oscillation")
    # Solution: Relax constraints or reduce Q
```

Decision Tree:

Is it chattering (high-freq)?

Yes → Increase R or decrease Q

No → Go to next question

Is model accurate?

No → Fix model parameters first

Yes → Go to next question

Is horizon long enough?

No → Increase N

Yes → Go to next question

Are constraints too tight?

Yes → Relax constraints

No → Add velocity/rate penalties

Example Fix:

```
python
```

```
# Before (oscillating)
```

```
Q = 1000
```

```
R = 0.01
```

```
N = 5
```

```
# After (stable)
```

```
Q = 100 # Reduced by 10×
```

```
R = 1.0 # Increased by 100×
```

```
N = 15 # Increased by 3×
```

```
S = 5.0 # Added rate penalty
```

Q10.2: Your MPC is too slow to run in real-time. What do you do?

Answer:

Step 1: Profile the Code

```
python
```

```

import time

def profile_mpc():
    start = time.time()

    # Time each component
    t1 = time.time()
    candidates = generate_candidates()
    print(f"Generate candidates: {(time.time()-t1)*1000:.1f}ms")

    t2 = time.time()
    for candidate in candidates:
        cost = evaluate(candidate)
    print(f"Evaluate all: {(time.time()-t2)*1000:.1f}ms")

    print(f"Total: {(time.time()-start)*1000:.1f}ms")

```

Step 2: Reduce Computation

Option 1: Shorter Horizon

```

python

# Before
N = 20 # 1 second lookahead at 50Hz

# After
N = 10 # 0.5 second (2× faster!)

```

Option 2: Fewer Samples

```

python

# Before
n_samples = 30 # Try 30 different controls

# After
n_samples = 10 # Try only 10 (3× faster!)

```

Option 3: Coarser Discretization

```

python

```

```
# Before
```

```
dt = 0.02 # 50 Hz
```

```
# After
```

```
dt = 0.05 # 20 Hz (but keep N the same for same lookahead)
```

Step 3: Optimize Code

Use NumPy Vectorization:

```
python
```

```
# SLOW: Python loops
```

```
cost = 0
```

```
for i in range(N):
```

```
    cost += Q * (state[i] - target)**2
```

```
# FAST: NumPy vectorization
```

```
errors = states - target
```

```
cost = Q * np.sum(errors**2) # 10-100× faster!
```

Precompute Constants:

```
python
```

```
# SLOW: Compute every time
```

```
for candidate in candidates:
```

```
    cost = (100.0 * error**2 +
```

```
           0.1 * control**2 +
```

```
           10.0 * constraint_penalty)
```

```
# FAST: Precompute
```

```
Q = 100.0
```

```
R = 0.1
```

```
for candidate in candidates:
```

```
    cost = Q * error**2 + R * control**2 # Slightly faster
```

Step 4: Use Compiled Code

Option A: Numba (JIT compilation)

```
python
```

```

from numba import jit

@jit(nopython=True) # Compile to machine code
def evaluate_control(state, control, target, N, dt):
    cost = 0.0
    for step in range(N):
        # ... compute cost
        cost += step_cost
    return cost

# First call: slow (compilation)
# Subsequent calls: 10-100× faster!

```

Option B: Cython

```

cython

# Write critical loop in Cython (C extension)
cdef double evaluate_control(double[:] state, double[:] control):
    cdef double cost = 0.0
    cdef int i
    for i in range(N):
        cost += Q * (state[i] - target)**2
    return cost

```

Option C: C++ with Python bindings

```

cpp

// Most performant option for real-time
// Write MPC in C++, call from Python

```

Step 5: Use Better Hardware

```

python

# Run on faster computer
# Raspberry Pi → Jetson Nano → Full PC

# Or: Use GPU
import cupy as cp # GPU-accelerated NumPy
# 10-1000× faster for large matrices

```

Step 6: Hierarchical Control

python

Slow outer loop (MPC)

```
def outer_loop(rate=1.0): # 1 Hz
    while True:
        reference = mpc.compute_setpoint()
        sleep(1/rate)
```

Fast inner loop (PID)

```
def inner_loop(rate=100.0): # 100 Hz
    while True:
        control = pid.track_reference()
        sleep(1/rate)
```

Timing Budget Example:

Control period: 50ms (20 Hz)

Budget allocation:

- Read sensors: 5ms
- MPC optimization: 35ms ← Must fit here!
- Send commands: 5ms
- Margin: 5ms

If MPC takes 40ms → TOO SLOW

Ultimate Solution: Code Generation

python

Use tools like CasADi or ACADO

Generate optimized C code offline

Achieves 0.1-1ms solve times!

```
import casadi
# ... define problem
solver = casadi.nlpsol('solver', 'ipopt', nlp)
# Generate C code
solver.generate('mpc_solver.c')
# Compile and run ultra-fast
```

Q10.3: Design an MPC for a system you've never seen before.

Interviewer gives: "Design MPC for a robotic arm that paints walls."

Your Answer (Structured Approach):

Step 1: Identify the System

Q: What are we controlling?

A: Robot arm position

Q: What can we change?

A: Joint torques (or motor voltages)

Q: What's the goal?

A: Follow a painting path on the wall

Step 2: Define State Variables

```
python

# State: What describes the system?
state = [
     $\theta_1$ ,  $\theta_2$ ,  $\theta_3$ ,      # Joint angles (position)
     $\omega_1$ ,  $\omega_2$ ,  $\omega_3$ ,  # Joint velocities
    x_ee, y_ee, z_ee # End-effector position (derived)
]

n_states = 9 # Or 6 if we only track joints
```

Step 3: Define Control Inputs

```
python

# Control: What can we command?
control = [
     $\tau_1$ ,  $\tau_2$ ,  $\tau_3$  # Joint torques
]

n_controls = 3
```

Step 4: Model the Dynamics

```
python
```

```

# Simplified robot arm dynamics
# (Real version would be complex)

class RobotArmModel(SystemModel):
    def dynamics(self, state, control, dt):
         $\theta$  = state[0:3]
         $\omega$  = state[3:6]
         $\tau$  = control

        # Simplified:  $\tau = I \cdot \alpha + \text{friction}$ 
         $\alpha = (\tau - b \cdot \omega) / I$  # Angular acceleration

         $\omega_{\text{next}} = \omega + \alpha \cdot dt$ 
         $\theta_{\text{next}} = \theta + \omega \cdot dt$ 

        # Forward kinematics for end-effector
        x_ee, y_ee, z_ee = forward_kinematics( $\theta_{\text{next}}$ )

        return np.concatenate([ $\theta_{\text{next}}$ ,  $\omega_{\text{next}}$ , [x_ee, y_ee, z_ee]])

```

Step 5: Define Constraints

```

python

# Joint limits (mechanical stops)
 $\theta_{\text{min}} = [-180^\circ, -90^\circ, -180^\circ]$ 
 $\theta_{\text{max}} = [180^\circ, 90^\circ, 180^\circ]$ 

# Torque limits (motor capacity)
 $\tau_{\text{min}} = [-50 \text{ Nm}, -50 \text{ Nm}, -30 \text{ Nm}]$ 
 $\tau_{\text{max}} = [50 \text{ Nm}, 50 \text{ Nm}, 30 \text{ Nm}]$ 

# Velocity limits (safety)
 $\omega_{\text{max}} = [90^\circ/\text{s}, 90^\circ/\text{s}, 90^\circ/\text{s}]$ 

# Collision avoidance (stay away from wall)
distance_to_wall > 0.05m

```

Step 6: Design Cost Function

```

python

```

```

def cost_function(state, control, target_path):
    # End-effector position
    x_ee, y_ee, z_ee = state[6:9]
    x_target, y_target, z_target = target_path

    # Cost components
    position_error = ((x_ee - x_target)**2 +
                      (y_ee - y_target)**2 +
                      (z_ee - z_target)**2)

    velocity_error = np.sum(state[3:6]**2) # Minimize velocity

    control_effort = np.sum(control**2)

    # Total cost
    J = (Q_pos * position_error +
         Q_vel * velocity_error +
         R * control_effort)

    return J

```

Step 7: Set Parameters

```

python

# MPC parameters (initial guess)
dt = 0.05      # 50ms (20 Hz control)
N = 20         # 1 second lookahead
Q_pos = 1000.0 # High - accurate painting important!
Q_vel = 10.0   # Moderate - some smoothness
R = 0.1        # Low - allow high torques if needed

# Tune after testing!

```

Step 8: Implementation Sketch

```

python

```

```

class PaintingRobotMPC:
    def __init__(self):
        self.model = RobotArmModel(I=0.5, b=0.1)
        self.mpc = MPC_Controller(self.model, dt=0.05, horizon=20)
        self.mpc.set_weights(Q=1000, R=0.1)

    def follow_path(self, painting_path):
        current_state = self.read_joint_sensors()

        for waypoint in painting_path:
            # MPC computes torques
            torques = self.mpc.compute_control(current_state, waypoint)

            # Apply to motors
            self.send_motor_commands(torques)

            # Update state
            current_state = self.read_joint_sensors()

            time.sleep(0.05)

```

Step 9: Validation Plan

1. Simulate without robot (software only)
2. Test with limited motion range
3. Test with slow speeds
4. Gradually increase speed/range
5. Add painting tool and test quality

Step 10: Expected Interview Follow-ups

Q: What if arm collides with wall? A: Add collision constraint in MPC cost:

```

python

if distance_to_wall < safe_distance:
    penalty += 10000 * (safe_distance - distance)**2

```

Q: How to handle different paint patterns? A: Just change target_path input - MPC automatically adapts!

Q: What if motors can't respond fast enough? A: Increase motor time constant in model, or reduce Q/increase R for smoother commands.

BONUS: Quick-Fire Questions

Q: What's the first step in implementing MPC?

A: Model the system dynamics (state equations).

Q: MPC vs PID in one sentence?

A: MPC predicts and optimizes future behavior; PID reacts to current error.

Q: Can MPC handle delays?

A: Yes! Include delay in the model: $x(k+d) = f(x(k), u(k-d))$

Q: What if I don't have a model?

A: Use data-driven methods (neural networks) or system identification.

Q: Biggest advantage of MPC?

A: Natural handling of constraints and multivariable systems.

Q: Biggest disadvantage?

A: Computational cost and need for accurate model.

Q: Is MPC suitable for safety-critical systems?

A: Yes, if properly validated and with safety backups. Used in aerospace, chemical plants.

Q: Can MPC learn online?

A: Yes - adaptive MPC updates model parameters in real-time.

Q: What's the difference between MPC and optimal control?

A: MPC is receding horizon optimal control (re-optimizes each step).

Q: Do you need a linear model?

A: No - nonlinear MPC exists, just slower to solve.

FINAL TIPS FOR INTERVIEWS

1. **Always start with fundamentals** - Explain the predict-optimize-apply cycle
2. **Use concrete examples** - Drone, car, temperature (things interviewer can visualize)
3. **Acknowledge trade-offs** - No free lunch (speed vs accuracy, etc.)

4. **Show practical thinking** - Mention real-world issues (timing, noise, constraints)
 5. **Be honest about limits** - "I'd need to research X" is better than making things up
 6. **Relate to their domain** - Connect MPC to their specific application
 7. **Draw diagrams** - Sketch the receding horizon, cost function, etc.
-

Good luck with your interview! 🚀